Analysis & Design Document

RPG Style Dialogue System

Game-Based Learning Study — Game-LLM-Interactive Condition

Valentino Clerx

Fontys Master of Applied IT

2026

---

1. Introduction

This document details the architectural design and logic of the RPG-style dialogue system used in the "Game-Hardcoded" experimental condition. The system is designed to act as a structured knowledge resource, providing scaffolded instruction and corrective feedback based on player interactions within the Feeding and Crafting module.

2. System Architecture

The system follows a Separation of Concerns (SoC) pattern, splitting the narrative data, the UI controller, and the gameplay observer into discrete units. This allows the dialogue logic to remain generic and reusable across different game modules.

2.1 Component Breakdown

- **DialogueManager.gd (The Controller):** Manages the state machine of the current conversation. It handles text parsing, the typewriter animation, character portrait swapping, and history tracking.
- **VetCraftingTrigger.gd (The Observer):** Monitors the Bowl.gd state. It acts as a bridge that translates physical game events (e.g., "Ingredient added to bowl") into narrative events (e.g., "Trigger node_L1_intro").
- **Dialogue Tree (The Data):** A nested Dictionary structure where each key is a unique node_id containing the response text and an array of branching options.

3. Design

3.1 Data Structure: Node-Based Tree

The dialogue is structured as a directed graph. Each node contains metadata that the DialogueManager interprets to update the UI state.

- **text**: The string displayed in the dialogue box.
- **options**: An array of dictionaries containing text (button label) and next (the ID of the target node).
- **back functionality**: A reserved keyword in the next field that instructs the manager to pop the previous ID from the history stack, enabling non-linear exploration.

3.2 Visual Logic: Typewriter & State

To ensure the system feels reactive, the character portrait (vet_character) is synchronized with the typing_tween.

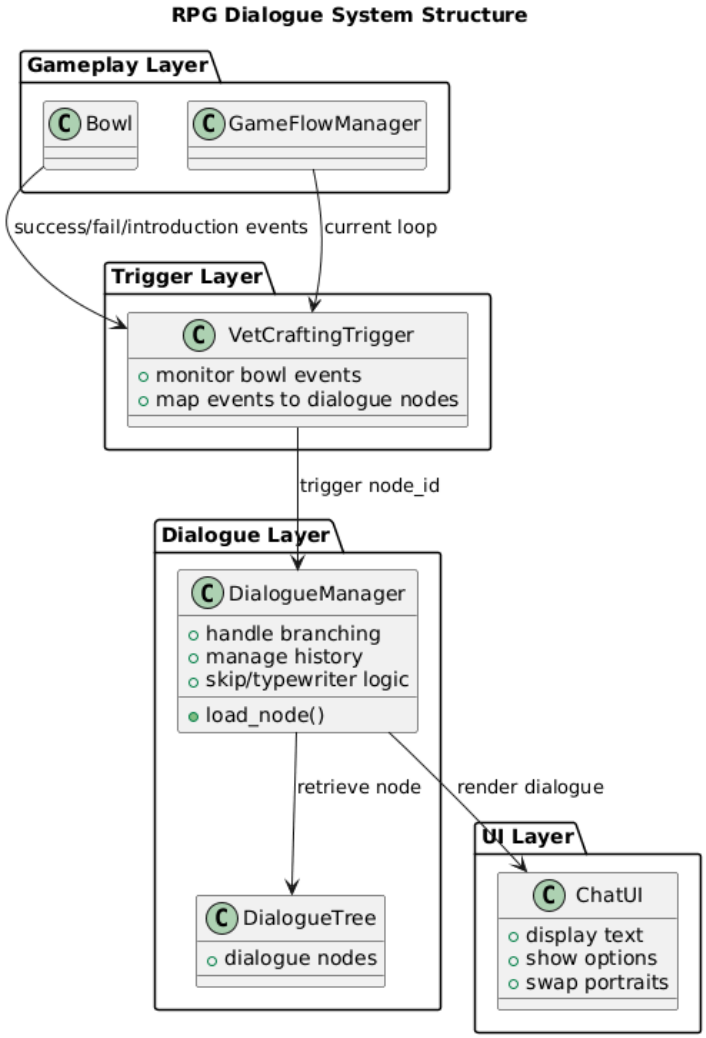
1. **Start**: Swap texture to "Talking" and begin the tween.
2. **During**: Increment visible_characters based on typing_speed.
3. **Finish**: Swap texture back to "Neutral" and call _show_options().

3.3 State Mapping (Triggers)

The system uses "Interceptive Logic" to ensure the NPC feels aware of the player's progress. The VetCraftingTrigger maps the game state to the dialogue tree as follows:

Game Event	***Condition***	***Dialogue Node***
First ingredient added	current_loop == 1	node_L1_intro
First ingredient added	current_loop == 2	node_L2_intro
Dish Validation Failed	current_loop == 3	node_L3_dishfail

3.4 System Overview



4. Interaction Flow Diagram

- **Signal Emission:** The Bowl emits recipe_failed.
- **Logic Delay:** The Trigger waits 0.4 seconds (to allow ingredient return animations to finish).
- **UI Activation:** chat_canvas.show() is called.
- **Node Loading:** DialogueManager fetches the relevant failure node and begins the typewriter effect.
- **Player Choice:** The player selects an option, either progressing to a deeper explanation or closing the menu.

5. Accessibility and UX

- **Skipping:** Players can bypass the typewriter effect by clicking, ensuring that fast readers or returning players are not frustrated by slow text.
- **History Stack:** By storing the history array, the system ensures that players who accidentally click the wrong option or want to revisit a fact can do so without restarting the entire conversation.
- **Audio Feedback:** The type_sound provides an auditory cue for text progression, which is a standard accessibility feature in RPG dialogue systems.